UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

Topic 2: Parallel and Scalable Data Processing

Part 1: Parallelism Basics

Ch. 9.4, 12.2, 14.1.1, 14.6, 22.1-22.3, 22.4.1, 22.8 of Cow Book

Topic 3 – Outline

- Basics of Parallelism
  - Task Parallelism; Dask
  - Single-Node Multi-Core; SIMD; Accelerators
- Basics of Scalable Data Access
  - Paged Access; I/O Costs; Layouts/Access Patterns
  - Scaling Data Science Operations
- Data Parallelism: Parallelism + Scalability
  - Data-Parallel Data Science Operations
  - Optimizations and Hybrid Parallelism

Roadmap

1

SIMD

Single Instruction, Multiple Data

2

Amdahl's Law

Theoretical speedup limits

3

GPU Architecture

Massively parallel hardware

4

GPU Programming

CUDA & Python (CuPy / Numba)

5

Scalability Challenges

Bottlenecks & real-world limits

What Is SIMD?

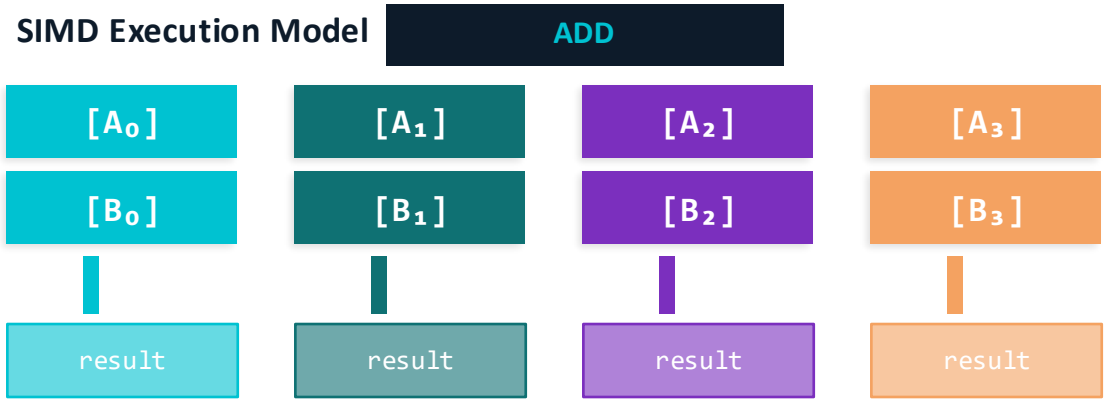
Definition

A processor executes ONE instruction simultaneously on MULTIPLE data elements — enabling data-level parallelism without explicit threading.

Flynn's Taxonomy

SISD → SIMD → MISD → MIMD

SIMD sits at the intersection of serial control flow and parallel data processing.



↑ *All 4 lanes processed simultaneously in one clock cycle*

Python

```python
import numpy as np
# NumPy uses SIMD under the hood
a = np.array([1.0, 2.0, 3.0, 4.0])
b = np.array([5.0, 6.0, 7.0, 8.0])
c = a + b  # → [6, 8, 10, 12] — vectorized!
```

Vector Registers & Python Impact

AVX-512 (Intel)

512-bit registers → process 16 × float32 or 8 × float64 simultaneously.

NumPy & SciPy

Compiled C/Fortran backends automatically generate SIMD instructions via compiler flags.

When to worry?

Pure Python loops CANNOT leverage SIMD. Always prefer vectorized NumPy over for-loops.

Scalar vs. Vectorized Performance

Python

```python
# ❌ Scalar — cannot use SIMD
def scalar_add(a, b):
    return [a[i] + b[i] for i in range(len(a))]

# ✅ Vectorized — SIMD exploited by NumPy
import numpy as np
def vector_add(a, b):
    return a + b   # triggers AVX/SSE instructions

# Benchmark
import timeit
n = 10000000
a, b = np.random.rand(n), np.random.rand(n)
t1 = timeit.timeit(lambda: scalar_add(a.tolist(), b.tolist()), number=1)
t2 = timeit.timeit(lambda: vector_add(a, b), number=1)
print(f"Scalar: {t1:.2f}s  |  Vectorized: {t2:.4f}s")
```

```
# Scalar: ~.6s  |  Vectorized: ~0.008s
```

SIMD Generalizations

- **Single-Instruction Multiple Thread (SIMT):** Generalizes notion of SIMD to *different threads* concurrently doing so
  - Each thread may be assigned a core or a whole PU
- **Single-Program Multiple Data (SPMD):** A higher level of abstraction generalizing SIMD operations or programs
  - Under the hood, may use multiple processes or threads
  - Each chunk of data processed by one core/PU
  - Applicable to any CPU, not just vectorized PUs
  - Most common form of parallel programming

Quantifying Efficiency: Speedup

Q: *How do we quantify the runtime performance benefits of multi-core parallelism?*

- As with task parallelism, we measure the speedup:

$$\textbf{Speedup} = \frac{\text{Completion time given only 1 core}}{\text{Completion time given n (>1) core}}$$

- In data science computations, an often useful surrogate for completion time is the instruction throughput: **FLOPs**, i.e., *number of floating point operations* per second.
- Data processing programs may run GFLOPs (10^9 FLOPs).
- Nowadays, even home user hardware (like higher-end graphics cards, gaming machines) touches TFLOP performance.

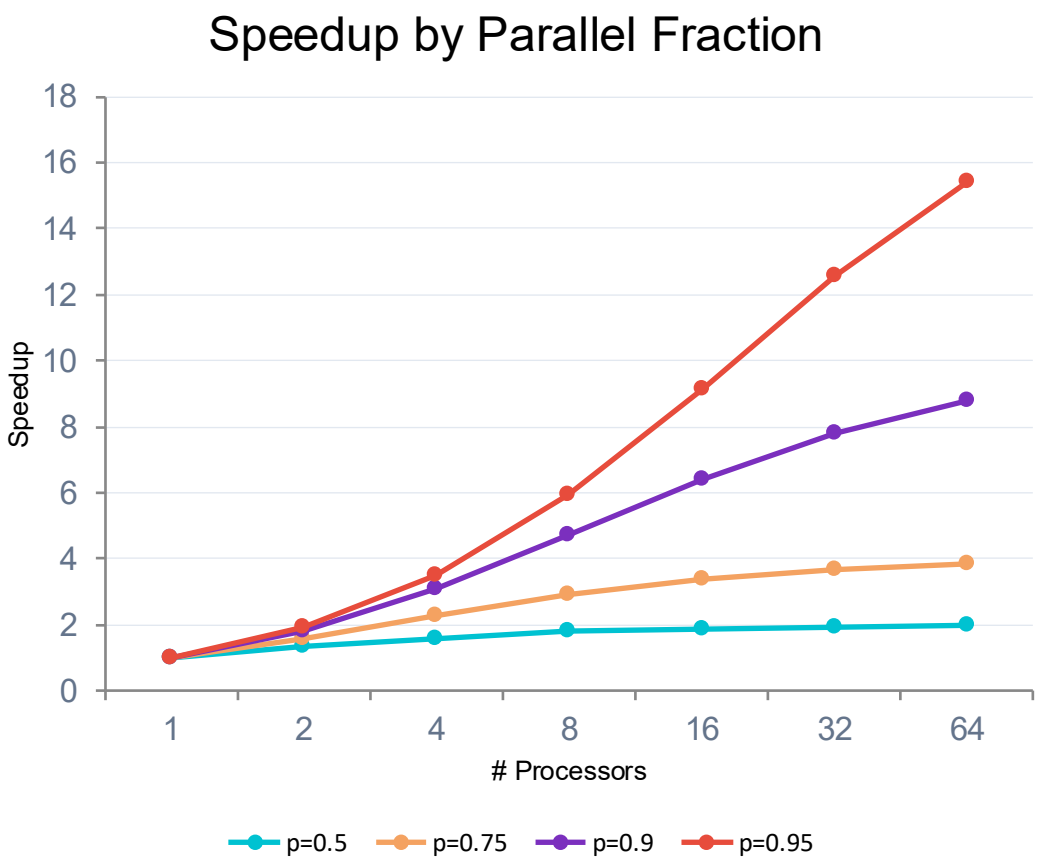
Can We Get N Speedup For N Cores?

$$\text{Speedup} = \frac{1}{[\,(1 - p) + p/n\,]}$$

p	Fraction of parallelizable work
n	Number of processors / cores
(1−p)	Serial bottleneck (cannot be parallelized)
Speedup	How much faster vs. single core

Key insight:
Even infinite processors
can't overcome the serial fraction!

Amdahl's Law: Speedup vs Processors



Python

```python
def amdahl_speedup(p, n):
    """Compute Amdahl speedup.
    p: parallel fraction (0-1)
    n: number of processors
    """
    return 1 / ((1 - p) + p / n)

# Max possible speedup (n→∞)
for p in [0.5, 0.75, 0.9, 0.95]:
    max_speedup = 1 / (1 - p)
    print(f"p={p:.2f} → max speedup: {max_speedup:.1f}x")

# Output:
# p=0.50 → max speedup: 2.0x
# p=0.75 → max speedup: 4.0x
# p=0.90 → max speedup: 10.0x
# p=0.95 → max speedup: 20.0x
```

Where Amdahl's Law Breaks Down

Amdahl assumes: **FIXED problem size**
=> **Serial fraction is a constant proportion of total work**

	Scenario A (Amdahl's world)	Scenario B (Fixed problem, 8 cores)	Scenario C (8× bigger problem, 8 cores)
Serial work	**1s**	**1s**	**1s**
Parallel work	**7s**	**7s → 0.875s**	**56s → 7s**
Cores	**1**	**8**	**8**
Wall time	**8s**	**1.875s**	**8s**
Actual speedup	**1×**	**4.3×**	**~8×**
	Baseline — single core	Amdahl predicts: 4.3× ✓ Correct	**Amdahl still predicts: 4.3×** **✗ Wrong!**

In Scenario C the serial overhead (1s) is now only 1/8 of its original proportion — Amdahl uses the wrong p!

Broken Assumption: Gustafson's Fix

Why Amdahl's Law Fails

Constant-time serial overhead

Many serial steps (open a file, initialize a model, merge outputs) take roughly the SAME wall-clock time regardless of data size. As data grows, their fraction of total work shrinks — Amdahl assumes it stays fixed.

Real workloads scale with hardware

When a team gets a 64-core cluster they don't re-run the same job — they process 64× more data in the same time window. Problem size and core count grow together.

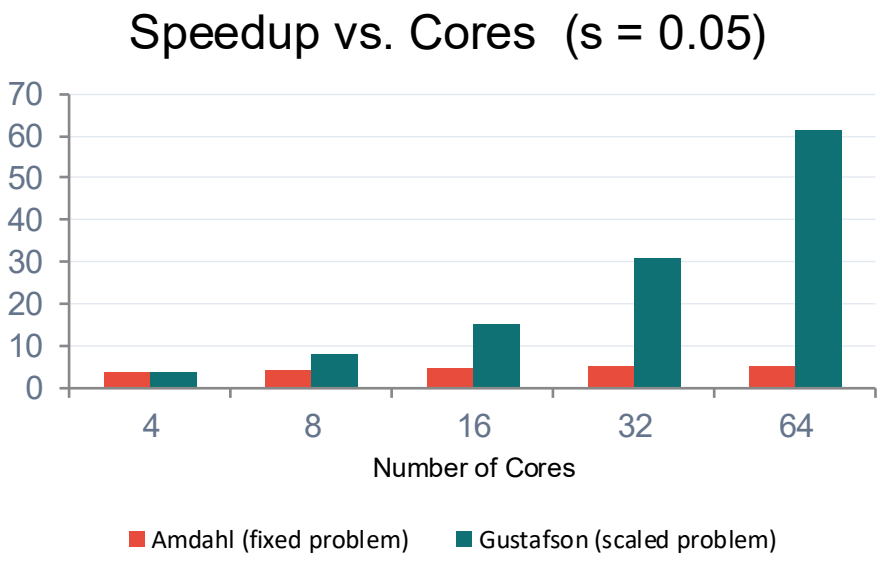
Amdahl's hidden assumption

The formula requires p to be fixed. In a scaled problem, p → 1 as data grows — making Amdahl too pessimistic.

Gustafson's Law (1988) — The Fix

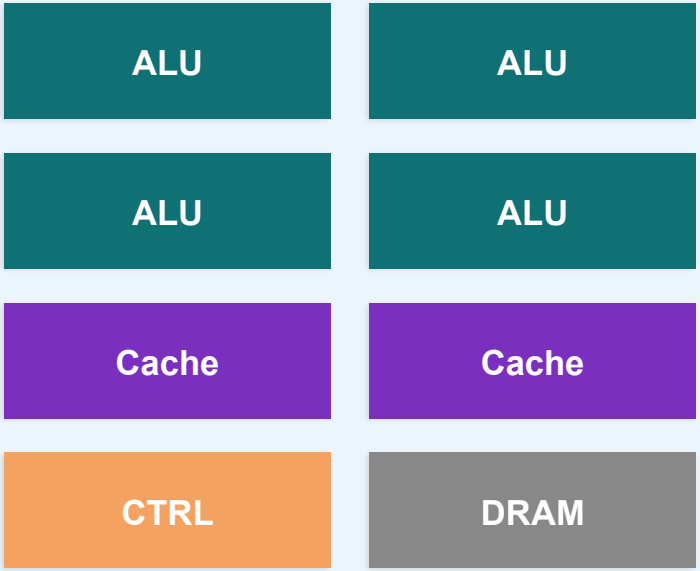
$$\text{Scaled Speedup} = n - s \times (n - 1)$$

n = cores | s = serial fraction of the SCALED job



CPU vs GPU: Architectural Philosophy

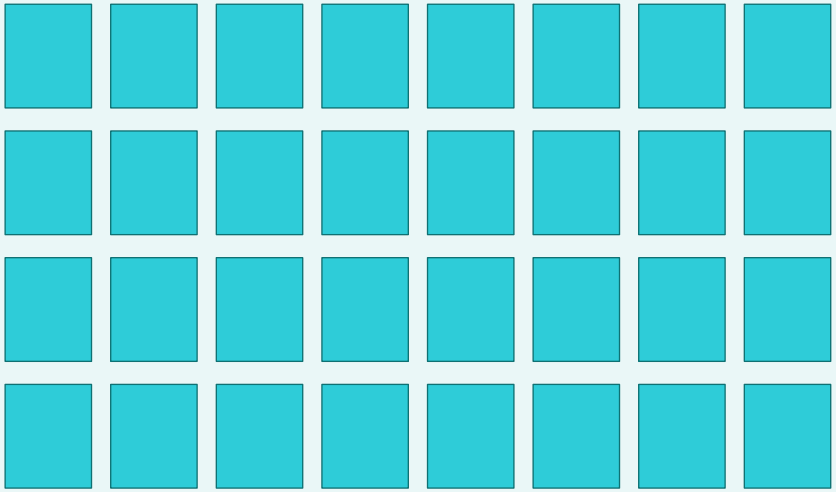
CPU



4–32 powerful cores
Optimized for low latency
Large caches, branch prediction

vs

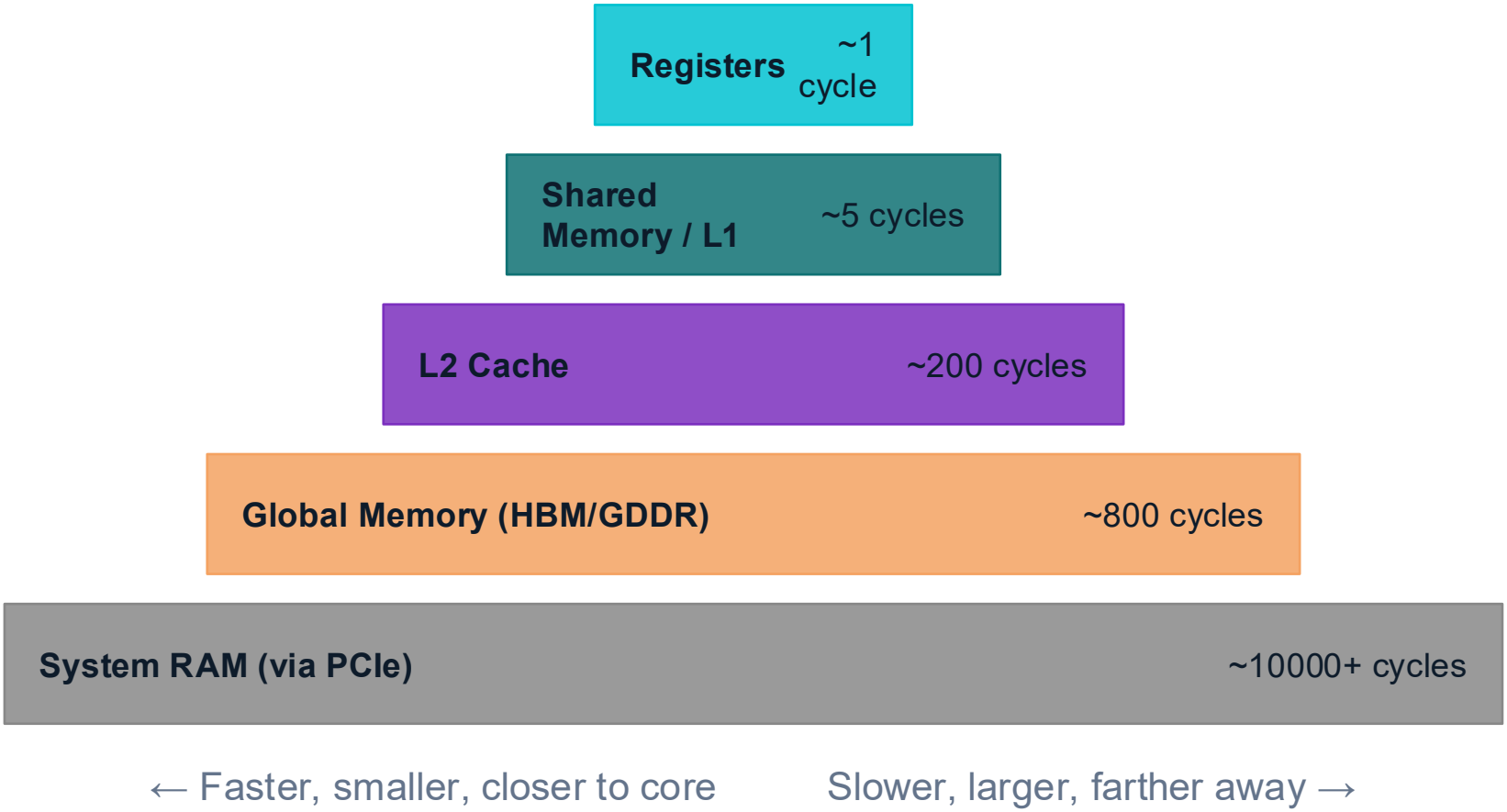
GPU



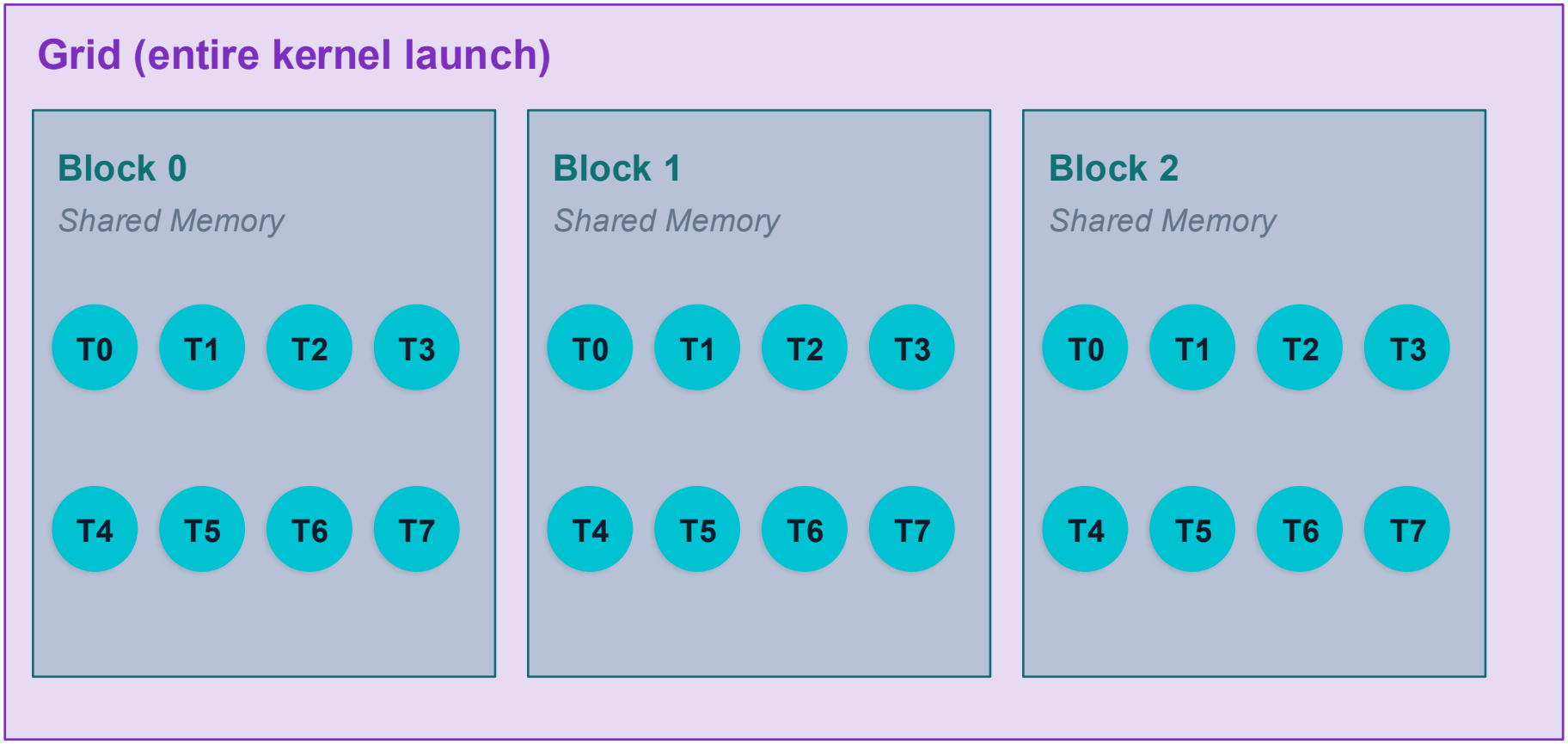
SM × 32 shown (real GPUs: 1000s)

1000s–10000s of small cores
Optimized for throughput
Many parallel threads

GPU Memory Hierarchy



CUDA Execution Model



Key: Grid → Blocks → Threads
Threads in a Block share memory
Warp = 32 threads executed together

CuPy: NumPy-Compatible GPU Arrays

What is CuPy?

A NumPy-compatible array library that runs entirely on the GPU. Almost identical API — just change `import numpy` to `import cupy`.

When to use?

Matrix operations, element-wise math, FFTs, sorting, reduction operations on large arrays (>1M elements).

Gotcha!

Data lives on GPU. Transferring back to CPU (cp.asnumpy()) incurs PCIe overhead — minimize transfers!

Python

```python
import numpy as np
import cupy as cp    # pip install cupy-cuda12x

# Move data to GPU
A_cpu = np.random.rand(10000, 10000).astype('float32')
A_gpu = cp.asarray(A_cpu)   # Host → Device

# Allocated DIRECTLY on the GPU. No Transfer
B_gpu = cp.random.rand(10000, 10000, dtype='float32')

# GPU matrix multiply — same API as NumPy!
C_gpu = cp.dot(A_gpu, B_gpu)   # runs on GPU

# Bring result back to CPU only when needed
C_cpu = cp.asnumpy(C_gpu)      # Device → Host

# Element-wise operations chained together.
Exponentiation and Squareroot. TWO SEPARATE
KERNELS ARE LAUNCHED.
D_gpu = cp.exp(A_gpu) + cp.sqrt(B_gpu)  # vectorized

print(f"Shape: {D_gpu.shape}, Device: {D_gpu.device}")
```

Numba: Python Custom CUDA Kernels

Numba @cuda.jit

Decorator that compiles Python functions to real CUDA PTX (*) code. No C++ required — write kernels in Python.

Thread indexing

Each thread gets its position via cuda.threadIdx, cuda.blockIdx. Use this to index into your data array.

Grid stride loops

Handle arrays larger than your grid with grid-stride loops — industry best practice.

Python

```python
from numba import cuda
import numpy as np

@cuda.jit
def vector_add_kernel(a, b, result):
    # Identify this thread's position
    idx = cuda.grid(1)   # = blockIdx.x * blockDim.x + threadIdx.x
    if idx < result.size:
        result[idx] = a[idx] + b[idx]

# Prepare data
n = 1_000_000
a = np.ones(n, dtype=np.float32)
b = np.ones(n, dtype=np.float32)
result = np.zeros(n, dtype=np.float32)

# Launch kernel: 1024 threads/block
threads_per_block = 1024
blocks = (n + threads_per_block - 1) // threads_per_block
vector_add_kernel[blocks, threads_per_block](a, b, result)

print(result[:5])  # [2. 2. 2. 2. 2.]
```

(*) PTX (Parallel Thread Execution) : A low-level, virtual machine ISA used in CUDA programming

GPU Coding: PyTorch, Deep Learning

Python

```python
import torch
import torch.nn as nn

# Check GPU availability
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using: {device}")
print(f"GPU: {torch.cuda.get_device_name(0)}")

# Move tensors to GPU with .to(device)
X = torch.randn(1024, 512).to(device)
y = torch.randint(0, 10, (1024,)).to(device)

# Model automatically runs on GPU
model = nn.Sequential(
    nn.Linear(512, 256), nn.ReLU(),
    nn.Linear(256, 10)
).to(device)

optimizer = torch.optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

# Training loop — everything stays on GPU
for epoch in range(100):
    pred = model(X)
    loss = loss_fn(pred, y)
    optimizer.zero_grad()
    loss.backward()       # Backprop on GPU!
    optimizer.step()
```

Tensor operations

All PyTorch ops (matmul, conv2d, softmax) dispatch to optimized CUDA kernels via cuBLAS / cuDNN.

DataLoader & GPU

Use `pin_memory=True` and `non_blocking=True` on `.to(device)` to overlap CPU→GPU transfer with computation.

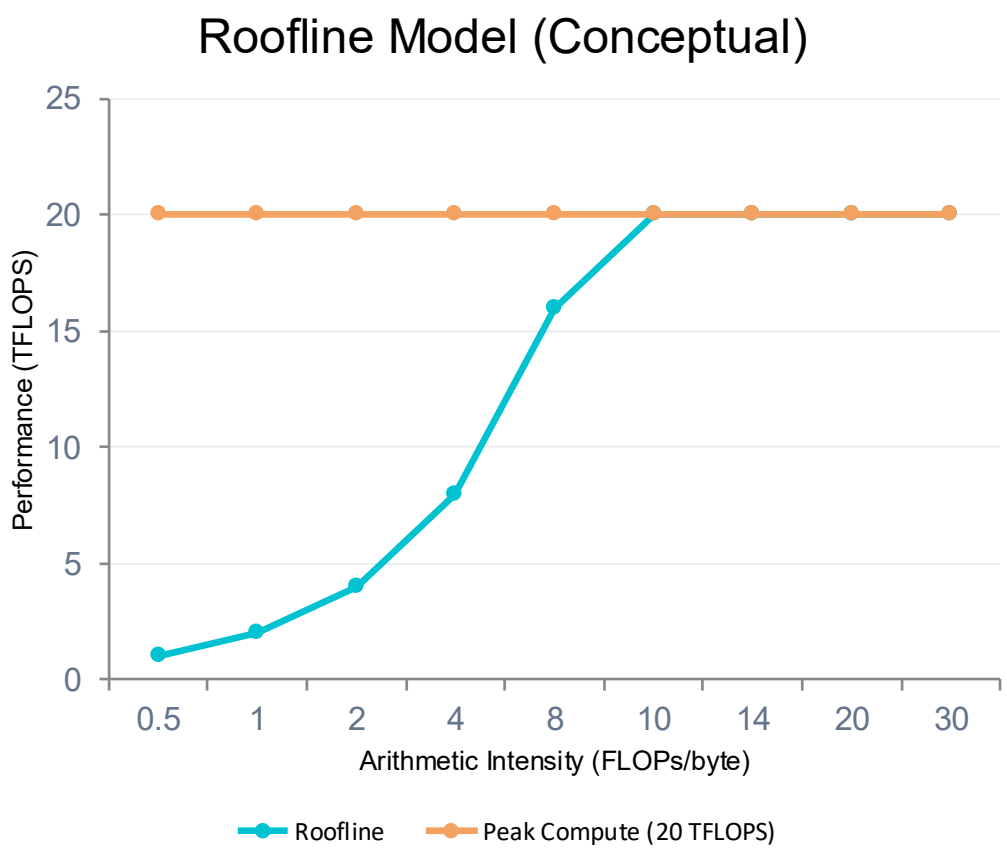
Multi-GPU

torch.nn.DataParallel or DistributedDataParallel splits batches across GPUs automatically.

Scalability Challenges & GPU Bottlenecks

The Roofline Model

GPU performance dependent on compute vs. memory-bound behavior



Memory-Bound Region

Left of ridge point: performance limited by memory bandwidth, not compute. Adding more cores does NOT help.

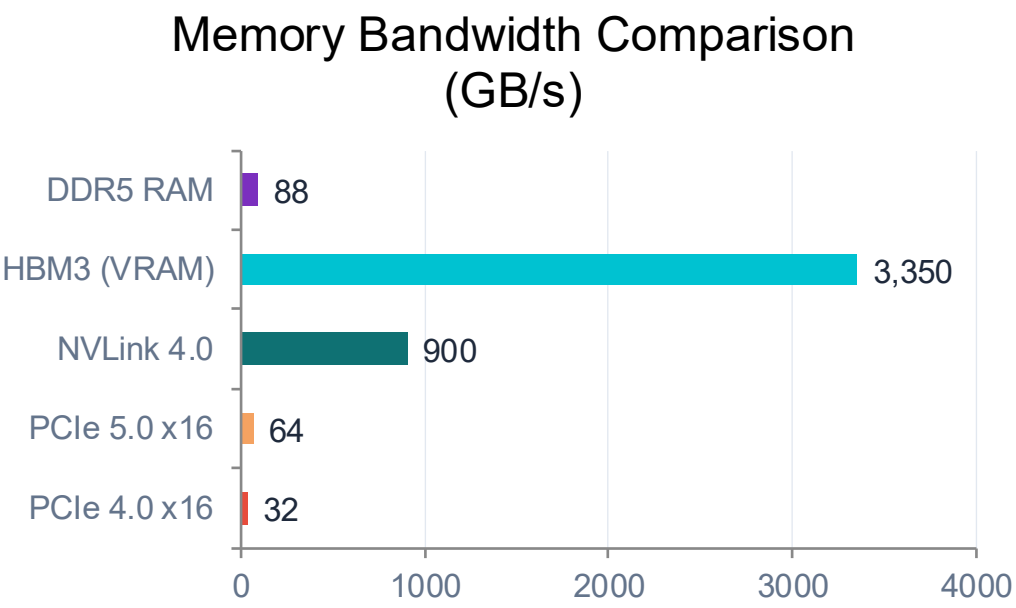
Compute-Bound Region

Right of ridge point: GPU FLOPs are the bottleneck. Optimize kernel efficiency.

Ridge Point

Optimal operational intensity.
Matrix multiply sits here — that's why matmul is GPU-efficient!

Bottleneck #1: PCIe Data Transfer



Best practice: Minimize data transfers—keep data on GPU as long as possible.

- Async transfers: use cuda.Stream() to overlap compute & transfer
- Prefetching: load next batch while current batch trains
- Pin memory: page-locked CPU memory is faster to transfer

Python

```python
import torch, time

def benchmark_transfer(size_gb=1.0):
    n = int(size_gb * 1e9 / 4)  # float32
    x_cpu = torch.randn(n)

    # CPU → GPU (Host to Device)
    t0 = time.perf_counter()
    x_gpu = x_cpu.cuda()
    torch.cuda.synchronize()
    h2d = (size_gb / (time.perf_counter() - t0))

    # GPU → CPU (Device to Host)
    t0 = time.perf_counter()
    _ = x_gpu.cpu()
    torch.cuda.synchronize()
    d2h = (size_gb / (time.perf_counter() - t0))

    print(f"H→D: {h2d:.1f} GB/s")
    print(f"D→H: {d2h:.1f} GB/s")

benchmark_transfer(1.0)
# H→D: ~14 GB/s  (PCIe 4.0 x16 limit)
```

Bottleneck #2: GPU Memory Issues

Be mindful of GPU memory capacity and utilization.

Python

```python
import torch

def profile_memory():
    print(f"Total VRAM:
{torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
    print(f"Allocated:      {torch.cuda.memory_allocated() / 1e9:.3f}
GB")
    print(f"Reserved:       {torch.cuda.memory_reserved() / 1e9:.3f}
GB")

# OOM error example — and how to handle it
try:
    giant = torch.randn(100000, 100000).cuda()  # ~40 GB!
except torch.cuda.OutOfMemoryError as e:
    print(f"OOM! {e}")
    torch.cuda.empty_cache()  # free unreferenced memory

# ✅ Mixed precision — halves memory usage
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()
with autocast():            # auto fp16/bf16
    output = model(input)
loss = criterion(output, target)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

OOM = Out of Memory

Most common GPU error.
Reduce batch size, use gradient checkpointing, or mixed precision (fp16).

Mixed Precision (AMP)

Store weights in fp32, compute in fp16. Cuts VRAM in half and doubles throughput on Tensor Cores!

Gradient Checkpointing

Recompute activations during backward pass instead of storing them. Trades compute for memory.

Bottleneck #3: GPU Threads Issues

In GPUs, warps are lockstepped groups of typically 32 threads allocated to a Streaming Multiprocessor (SM). Avoid thread divergence and low occupancy.

Thread Divergence

Threads in a warp (32) must execute the SAME instruction. If-else causes serial execution of both branches — devastating for performance.

Occupancy

Ratio of active warps to max possible warps per SM. Low occupancy = GPU is underutilized. Target >50%.

Profiling Tools

NVIDIA Nsight Systems & Compute. Python: torch.profiler for PyTorch workloads.

Python

```python
# ❌ Divergent kernel — avoid in CUDA
@cuda.jit
def bad_kernel(x, out):
    i = cuda.grid(1)
    if i < x.size:
        if x[i] > 0:     # ← threads in warp diverge!
            out[i] = x[i] ** 2
        else:
            out[i] = -x[i]

# ✅ Use math ops instead of branches
@cuda.jit
def good_kernel(x, out):
    i = cuda.grid(1)
    if i < x.size:
        pos = max(x[i], 0.0) ** 2   # no branch!
        neg = min(x[i], 0.0) * -1.0
        out[i] = pos + neg

# Profile with PyTorch
with torch.profiler.profile(
    activities=[torch.profiler.ProfilerActivity.GPU]
) as prof:
    model(input)
print(prof.key_averages().table(sort_by='cuda_time_total')
)
```

Multi-GPU Scaling Challenges

Communication Overhead

All-reduce operations between GPUs (gradient sync) scale as O(n) — can dominate at large scale.

Load Imbalance

Uneven data distribution leaves some GPUs idle. Use proper sharding and dynamic batching.

Python

```python
# Data Parallel — easiest
model = nn.DataParallel(model)  # uses all GPUs

# Distributed Data Parallel — more efficient
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
dist.init_process_group('nccl')
model = DDP(model, device_ids=[local_rank])
```

NCCL Collective Ops

NCCL (*) all-reduce is the most expensive op in distributed training. Overlap with backward pass using techniques like bucket gradient fusion.

Model vs Data Parallelism

Data parallelism: each GPU has full model, different data. Model parallelism: model split across GPUs (needed when model > VRAM).

(*) NCCL ("Nickel") is the NVIDIA Collective Communication Library.

- Provides highly optimized primitives (e.g., all-gather, all-reduce)
- High-bandwidth, low-latency inter-GPU communication.

GPU Optimization Checklist

1. **Profile first**
   Never optimize blindly — use Nsight / torch.profiler to find the real bottleneck

2. **Maximize arithmetic intensity**
   Structure kernels to reuse data from shared memory / registers (roofline model)

3. **Minimize PCIe transfers**
   Keep data on GPU; use async streams to overlap transfer with compute

4. **Use mixed precision (fp16/bf16) (*)**
   2× memory savings + 2–4× throughput on Tensor Cores

5. **Batch appropriately**
   Large batches maximize GPU utilization; small batches leave SMs idle

6. **Check Amdahl's serial fraction**
   No amount of GPU scaling helps if 20%+ of your pipeline is serial Python

7. **Prefer vectorized ops**
   CuPy / NumPy-style ops > explicit loops; avoid Python-level iteration on GPU data

(*) bf16 (Brain Float 16) is a ML-oriented 16-bit data format.

- Offers 8 exponent bits as fp32, but lower precision (7-bit mantissa)
- Improved training stability over fp16 (compared to fp16, can represent larger numbers without overflowing)

Key Takeaways

SIMD	Vectorize with NumPy — SIMD is free performance if you avoid Python loops
Amdahl	Serial fractions dominate at scale. Measure p before adding hardware
GPU	GPUs excel at throughput, not latency — thousands of simple parallel ops
Code	CuPy ≈ NumPy on GPU; Numba for custom kernels; PyTorch for ML pipelines
Bottleneck	PCIe transfer, memory capacity, thread divergence — know your bottleneck first

Further related reading: NVIDIA CUDA Docs, CuPy Docs, Numba CUDA Docs, PyTorch Profiler.